

# 第044讲 广义线性模型实验

这份实验从纸笔的三行数据走到固定的560行模拟。目标是证明自己实现了所写的Poisson目标，知道曝光量怎样进入，并能用正确的条件方差概念解释过度离散。先完成预测，再看程序输出；不要为了得到期待结果换种子。

直接先修为020、023、029、042至043。本讲数据全为原创合成数据。实验不涉及真实人员或业务决策，也不以本例名次宣布模型普遍优劣。

## 1 准备独立环境和目录

保留本单元目录完整，尤其是 data/、三个数据文件及校验文件。建议在Anaconda Prompt或终端进入 044 目录后执行：

```
conda env create -f environment.yml
conda activate ml044-glm
python --version
```

也可以创建Python3.12虚拟环境，再执行 `python -m pip install -r requirements.txt`。固定数值依赖为NumPy2.3.5、SciPy1.17.0、scikit-learn1.8.0、Matplotlib3.10.8。Notebook依赖另列在环境文件中。PDF重建另外使用WeasyPrint、Markdown、Node.js、MathJax及中文字体；依赖见build-requirements.txt与package.json。安装Python构建依赖后执行`npm install`即可准备本地公式渲染。只做科学实验不需要重建PDF。

在受限环境中，把临时文件与绘图缓存指向自己有写权限的目录。例如Linux或macOS终端：

```
mkdir -p outputs/tmp outputs/mpl outputs/cache
export TMPDIR="$PWD/outputs/tmp"
export MPLCONFIGDIR="$PWD/outputs/mpl"
export XDG_CACHE_HOME="$PWD/outputs/cache"
export OPENBLAS_NUM_THREADS=1
```

程序在未指定时设置 `LOKY_MAX_CPU_COUNT=1`，避免某些隔离运行环境探测物理核数失败。这是资源设置，不修改数据或统计目标。Windows可在Anaconda Prompt中设置对应环境变量，也可只运行科学脚本；本次实际运行不代表已验证Windows或Anaconda安装过程。

## 2 在运行前写下协议

打开 data/protocol.json，抄下种子4402026、320训练行、240测试行、真参数 (0.2, 1.1)、Gamma形状2和无正则设置。解释以下三件事：

- 为什么真实均值列不能放入训练特征？

- 为什么两种计数情景共享特征，并不等于可以把两列观测拼接来翻倍样本？
- 为什么知道真实生成机制之后选择一个教学模型，不等于用封存测试标签搜索模型？

本实验预声明的是三种固定候选，不从它们中反复选择新参数。所有测试指标一次性报告，不拿测试结果调学习率、链接、正则或种子。若以后设计新的实验，应另存数据和协议，不覆盖这一份。

### 3 先完成三个手算状态

建立含以下列的纸笔表：行号、 $x, y, \eta, \mu, \ell, \partial \ell / \partial \mu, \partial \mu / \partial \eta, \partial \ell / \partial \eta, c_b, c_w$ 。初始  $(b, w) = (0, 0)$ ，学习率0.2；三行  $x = (-1, 0, 1), y = (0, 1, 3), e = 1$ 。

第一遍只算前向。注意第三行的完整损失为  $1 + \log 6$ 。第二遍把局部导数连起来，平均梯度贡献要除以3。三个样本都算完后，才更新两个参数。

第一步应得到  $(1/15, 1/5)$ 。不要看到这一行就跳过第二轮：重新算三个指数、三个完整损失、三条导数链，得到  $(0.116685493543, 0.371304543132)$ 。最后再做第三次前向，平均损失应约 1.246373395。

把自己算的第2行三个损失写在一行，思考为何它逐渐上升而总损失下降。若你每行都更新参数，结果会不同；先检查算法顺序，而不是调整学习率强行匹配答案。

### 4 运行脚本并定位输出字段

```
python experiment.py --out outputs/result.json
python -0 experiment.py --out outputs/result-0.json
python audit.py --out outputs/audit.json
python -0 audit.py --out outputs/audit-0.json
```

主程序先核验三份数据输入的SHA256。-0 会去除Python的普通断言，因此必需校验全部用显式异常实现；这两条路径都应工作。科学脚本不覆盖教程、源码或归档数据；在本单元内，JSON只允许写入 outputs/。

在 tiny\_ledger 找到三个状态。逐一对照 eta、mu、loss、mean\_loss、三个局部导数字段、逐行 mean\_gradient\_contributions、平均 gradient 与 hessian。不是只比较最后的参数。若第一次就不一致，按“前向、局部导数、除以n、汇总、同步更新”的顺序查错。

tiny\_fit 是同目标的保护Newton完整拟合，它与两步梯度下降属于不同运行，不能把它的最终参数塞进手算过程。看 status 和 history: gradient\_tolerance 表示满足数值标准，max\_updates 表示预算用尽，其他失败状态要保留。行号0代表初始状态。

## 5 核验科学公式而不是只看程序退出

`audit.py` 包含可直接阅读的独立路径：

1. 使用80位Decimal，从标量Poisson概率质量求负对数，重算三次前向、梯度与Hessian。
2. 用得分方程导出的  $t = (3 + \sqrt{37})/2$  求主例有限MLE。
3. 使用实际SciPy `poisson.logpmf` 与sklearn `mean_poisson_deviance` 比较分布目标。
4. 在三个非驻点、步长  $10^{-3}, 10^{-4}, 10^{-5}$  下做中心差分，分别核验梯度和Hessian。
5. 用实际 `PoissonRegressor(alpha=0)` 对曝光加权率拟合，再把参数代回原计数目标计算梯度。

记录差分误差随步长的变化，不要求步长越小就永远越准确。过大的步长有截断误差，过小的步长会放大浮点相减误差。比较库结果前必须核对是否带截距、是否正则、样本权重如何归一化。

`audit.json` 的通过只涉及列出的作者数值与接口检查，不是统计假设已成立，也不等于独立课程验收。不要把测试项数量当作质量本身。

## 6 运行真实Notebook

启动Jupyter Lab后打开 `experiment.ipynb`，选择Python3内核，执行“Restart Kernel and Run All”。首个代码格只用标准库核验输入与核心实现文件，校验成功以后才导入数值库。这样不能沿用旧内核里的旧函数而不自知。

Notebook包含实际代码格、运行计数、文本输出与八幅真实PNG输出。先按顺序运行，再选择性改动副本。保留原教程Notebook作为参考，不要跳过失败格继续宣称成功。

如果环境不支持套接字连接，可用附带的受限环境执行器：

```
python execute_notebook.py --out outputs/executed.ipynb
python -O execute_notebook.py --out outputs/executed-0.ipynb
```

它在当前新Python进程中启动真实IPython InProcessKernel，按顺序执行所有代码格并保存新Notebook，不覆盖交付Notebook。该路径确实执行Python与IPython代码，但不测试浏览器界面、远程socket传输或其他内核。输入契约保护固定课程，不是恶意篡改的安全认证；修改代码开展新研究时，应在副本中记录新的契约。

## 7 比较计数模型并保留负面结果

在 `cases.poisson` 读取三种模型的 `test_metrics`。先验证样本数240一致，再抄MSE、MAE、非正预测数和deviance。不加offset的模型也要报告，不能只保留最好的一列。

线性基线并非不懂曝光量：它拟合  $y \approx e(b + wx)$ ，通过 `np.linalg.lstsq(eX,y)` 求解。测试有53个非正预测，因此全测试集Poisson deviance未定义。MSE与MAE仍可照常计算，程序不删除这些行。

讨论一个常见陷阱：若把负预测改为  $10^{-8}$  再评分，实际上新增了一个“线性加截断”模型，并且引入任意阈值。可以另立协议研究它，但不能悄悄用这种结果替换原模型。

重新生成图，不覆盖交付图：

```
python plots.py --report outputs/result.json --directory outputs/figures
```

图5的纵轴是率，评分表是次数。指出从一个尺度换到另一个尺度需要乘还是除曝光量，解释散点的非整数值。

## 8 两个曝光量不变性实验

实验A：固定一组已拟合参数和  $x = 0$ ，把曝光量从0.5变为2。预测次数应乘4，单位率不变。对遗漏曝光量的模型重复一次，指出它为什么不能响应观察时长变化。

实验B：单位改写，而物理观察不变。把所有曝光数值乘4，同时将截距减  $\log 4$ ，斜率不变。逐行预测、完整loss、梯度与Hessian应一致到浮点容差。

区分A与B：A真的增加观察时长；B只改变计量单位。audit.py 自动检查B，但请自己写出代数等式  $4e \exp(b - \log 4 + wx) = e \exp(b + wx)$ 。

## 9 检查条件过度离散

打开 cases.overdispersed。先读取原始训练均值与方差，再看Poisson拟合后的Pearson离散度。计算两种情景的原始方差均值比，解释为什么Poisson情景的比值也远大于1。

用  $\mu + \mu^2/2$  计算  $\mu = 1, 4, 10$  时的混合模型条件方差，并与Poisson方差比较。此处“过度”是相对于Poisson在相同条件下的方差约束，不是说原始数据的某个标准差太大。

本次混合情景的训练离散度约2.355146，测试Poisson平均deviance约2.847883。两个数不是同一个量，不能互换，也不应以“测试deviance必须为1”当作精确检验。不要因均值拟合还不错就输出未经验证的Poisson置信区间。

## 10 有针对性地验证失败路径

数学模型和教学实现支持域不同。先记录正文给出的数值范围，再在Notebook或独立Python副本中尝试：

- 把一个计数改成-1或0.5：应明确拒绝，不取整
- 把曝光量改成0：应拒绝，不偷偷加小数
- 用重复常数列拟合：应报告秩亏限制
- 令全部计数为0：应指出本例带截距模型无有限MLE
- 把更新预算设为0：应返回 max\_updates，不是伪造收敛

不要把这些临时改动写回归档CSV。audit.py 已含相同机制的代表性检查；继续堆几百个相似输入不能替代对统计目标的检查。保护不依赖 assert，因此普通与 -0 的结果都要核对。

## 11 数据和PDF的可恢复重建

```
python generate_data.py --out outputs/regenerated.csv
python build_pdf.py --directory outputs/pdfs
```

生成器只接受新输出路径，固定按x、曝光量、Poisson计数、Gamma乘子、混合Poisson计数顺序抽样。实验包中的CSV是权威输入；相同已验证版本可核对字节，其他平台或版本以明示数值容差与统计含义判断，不承诺任意环境字节相同。

PDF来自本单元三个Markdown和八张PNG，不需要其他讲的构建脚本。重建后用PDF阅读器检查所有页：中文、公式、表格、图注与页码，不能只检查文件存在或文本提取成功。

## 12 最终实验记录

提交一份简短记录，至少包含：三状态逐行手算；完整目标与参数顺序；独立参考与最大误差；实际停止状态；两种情景各三模型的完整测试表；曝光量两种实验；条件离散度解释；支持域和未验证范围。最后用四句话分别说明数值实现正确、优化已达到标准、统计假设是否合理、模拟结论能迁移到哪里。

若某一项失败，保留原结果，写明阻塞在哪里。这里没有通过换种子、删行或改协议来“修好”统计结论的步骤。